

Работа с Git через терминал

1. Выбор инструмента

Для задания выбран терминал Git в Windows. Основные команды проверены в отдельной учебной папке.

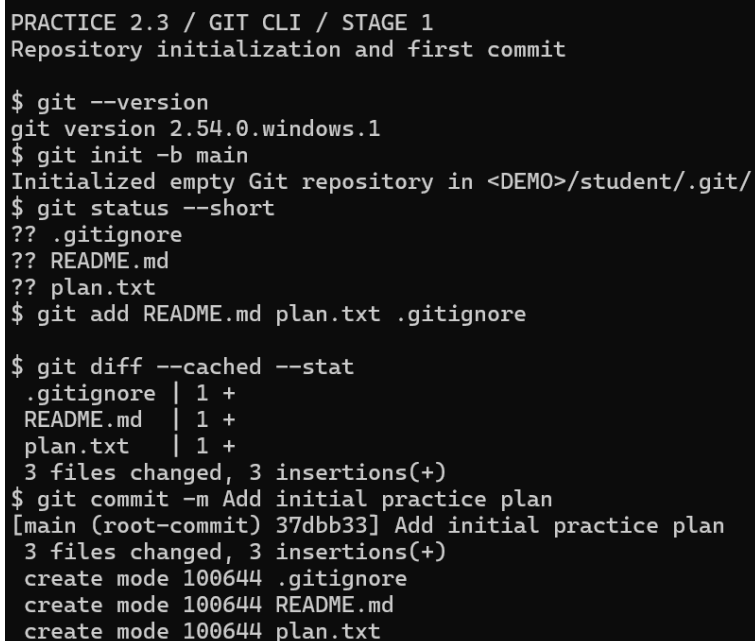
2. Создание репозитория и первый коммит

Команда `git init -b main` создала репозиторий. Подготовлены README.md, plan.txt и .gitignore с правилом *.tmp для исключения временных файлов.

Затем выполнены команды:

```
git status --short
git add README.md plan.txt .gitignore
git diff --cached --stat
git commit -m "Add initial practice plan"
```

Status показал новые файлы, add подготовил их к сохранению, а commit создал первый коммит (см. рис. 1).



```
PRACTICE 2.3 / GIT CLI / STAGE 1
Repository initialization and first commit

$ git --version
git version 2.54.0.windows.1
$ git init -b main
Initialized empty Git repository in <DEMO>/student/.git/
$ git status --short
?? .gitignore
?? README.md
?? plan.txt
$ git add README.md plan.txt .gitignore

$ git diff --cached --stat
.gitignore | 1 +
README.md | 1 +
plan.txt | 1 +
3 files changed, 3 insertions(+)
$ git commit -m Add initial practice plan
[main (root-commit) 37dbb33] Add initial practice plan
3 files changed, 3 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 plan.txt
```

Рисунок 1. Создание репозитория и сохранение файлов

3. Работа с ветками и разрешение конфликта

Командой `git switch -c feature/report` создана ветка, в которой изменён plan.txt и сделан коммит. После `git switch main` та же строка изменена иначе и сохранена в основной ветке.

При выполнении `git merge feature/report` возник конфликт: Git не смог автоматически объединить две версии одной строки (см. рис. 2). Команда `git status --short` показала UU plan.txt.

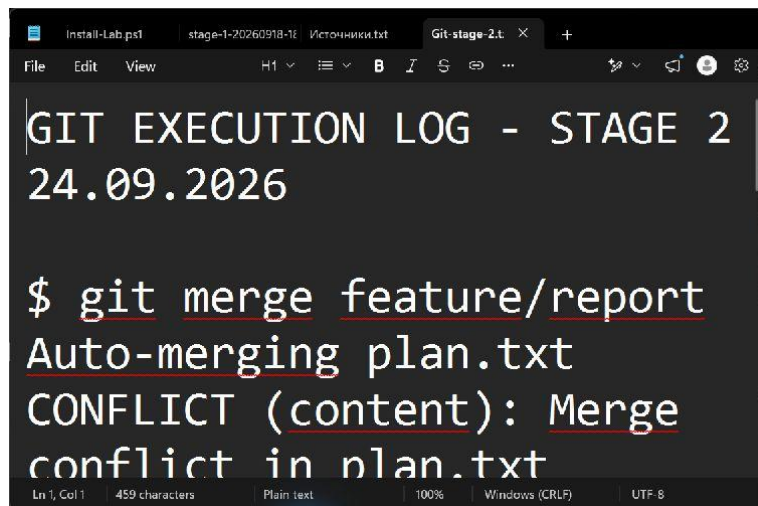
A screenshot of a code editor window with a dark theme. The title bar shows several tabs: 'Install-Lab.ps1', 'stage-1-20260918-1f', 'Источники.txt', and 'Git-stage-2.t'. The editor displays the text 'GIT EXECUTION LOG - STAGE 2' followed by the date '24.09.2026'. Below this, a command is shown: '\$ git merge feature/report'. The output indicates a conflict: 'Auto-merging plan.txt' followed by 'CONFLICT (content): Merge conflict in plan.txt'. The status bar at the bottom shows 'Ln 1, Col 1', '459 characters', 'Plain text', '100%', 'Windows (CRLF)', and 'UTF-8'.

Рисунок 2. Сообщение о конфликте в сохранённом журнале Git

В файле оставлена итоговая строка `review=checked-and-ready`, маркеры конфликта удалены. Затем выполнены `git add plan.txt` и `git commit -m "Resolve review plan conflict"`. Изменения обеих веток объединены.

4. Отмена изменений и обмен между репозиториями

Для проверки восстановления в `plan.txt` внесена временная правка. Команда `git restore plan.txt` отменила её. Затем создан коммит с экспериментальной строкой в `README.md`, а `git revert --no-edit HEAD` отменил этот коммит новым коммитом. История при этом сохранилась.

Для обмена использован локальный репозиторий `origin.git`. Он создан командой `git init --bare --initial-branch=main origin.git`. Основные команды обмена:

```
git remote add origin ../origin.git
git push -u origin main
git clone origin.git review
```

Clone выполнен из общей папки стенда, `remote add` и `push` - из первой копии. Во второй копии добавлен `review.txt`, выполнены коммит и `push`. В первой копии `git fetch origin` получил сведения об изменениях, `git pull --ff-only` обновил ветку. Публикация на GitHub не выполнялась.

5. Результат и оценка инструмента

В первой копии появился review.txt. Git status сообщил: nothing to commit, working tree clean. История проверена через git log --graph --oneline --all --decorate, готовое состояние отмечено командой git tag v1.0.

Достоинства терминала - точное управление операциями, подробный вывод и возможность повторять команды в скрипте. Недостатки - необходимость помнить команды и внимательно проверять текущую ветку и папку. Для выполнения основных действий терминал оказался удобным, особенно когда нужен журнал работы.

Журнал сохранён в terminal-session.txt, история - в practice-history.bundle. Скрипт reproduce_demo.py повторяет работу. Первый снимок относится к другому запуску, поэтому хеши отличаются; <DEMO> обозначает временную папку.

Источники: Chacon S., Straub B. Pro Git, разделы о сохранении изменений и ветвлении; локальная справка Git. Дата обращения: 24.09.2026.

Pro Git - сохранение изменений

Pro Git - ветвление и слияние